

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ
Anul 2013 - Informatică

Cuprins

I. Rezolvare Definitivat Varianta 3 (18 iulie 2013)	2
SUBIECTUL I (30 de puncte)	2
1. Metoda Greedy	2
2. Structura sistemelor de calcul: Unitatea Centrală de Procesare (UCP)	2
3. Programare: Perechi de numere prietene (amicabile)	2
SUBIECTUL al II-lea (30 de puncte)	4
1. Mijloc de învățământ pentru conținutul A: Subprograme recursive	4
a) Caracteristici și avantaje	4
b) Exemplu de valorificare didactică	4
2. Test practic pentru conținutul A: Subprograme recursive	5
SUBIECTUL al III-lea (30 de puncte)	5
Finalitățile educației	5
II. Rezolvare Definitivat Informatică de gestiune Varianta 3 (18 iulie 2013)	6
SUBIECTUL I (30 de puncte)	6
1. Formulare în produse pentru gestiunea bazelor de date	6
2. Unitatea centrală de procesare	6
3. Subprogramul sumap și verificarea numerelor prietene	6
4. Bază de date pentru companie de automobile	7
SUBIECTUL al II-lea (30 de puncte)	7
1. Mijloc de învățământ pentru subprograme recursive	7
2. Test practic pentru subprograme recursive	8
SUBIECTUL al III-lea (30 de puncte)	8

I. Rezolvare Definitivat Varianta 3 (18 iulie 2013)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Metoda Greedy

- **Principiu:** Metoda Greedy rezolvă probleme de optimizare în care soluția se construiește pas cu pas. La fiecare pas, se alege elementul cel mai optim local (care pare cel mai bun în acel moment), sperând că această succesiune de alegeri locale va conduce la un optim global. Nu se revine niciodată la alegerile deja făcute.
- **Oportunitate:** O problemă are proprietatea de alegere Greedy dacă un optim global poate fi obținut prin alegeri locale optime.
- **Problemă (Problema Rucsacului - varianta fracționară):**

► **C++:**

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct Obiect {
    double greutate, valoare, raport;
};
bool cmp(Obiect a, Obiect b) { return a.raport > b.raport; }
double greedyKnapsack(vector<Obiect> &obiecte, double capacitate) {
    sort(obiecte.begin(), obiecte.end(), cmp);
    double valoare_totala = 0.0;
    for (auto &obj : obiecte) {
        if (capacitate >= obj.greutate) {
            capacitate -= obj.greutate;
            valoare_totala += obj.valoare;
        } else {
            valoare_totala += capacitate * obj.raport;
            break;
        }
    }
    return valoare_totala;
}
```

2. Structura sistemelor de calcul: Unitatea Centrală de Procesare (UCP)

- **Componente:** Unitatea Aritmetico-Logică (UAL), Unitatea de Control (UC), Regiștrii de memorie interni.
- **Funcții:**
 1. **Faza de decodificare:** Identificarea instrucțiunii ce urmează a fi executată.
 2. **Faza de execuție:** Efectuarea operațiilor aritmetice sau logice corespunzătoare.
 3. Gestiunea fluxului de date între memoria principală (RAM) și dispozitivele periferice.

3. Programare: Perechi de numere prietene (amicabile)

Soluție C++:

```
#include <iostream>
#include <fstream>
using namespace std;

void sumap(long long n, long long &s) {
    s = 1;
    for (long long d = 2; d * d <= n; ++d) {
        if (n % d == 0) {
            s += d;
            if (d * d != n) {
                s += n / d;
            }
        }
    }
}

int main() {
    ifstream fin("DATE.TXT");
    long long a, b, sa, sb;
    fin >> a >> b;
    fin.close();

    sumap(a, sa);
    sumap(b, sb);
    if (sa == b && sb == a)
        cout << a << " " << b;
    else
        cout << "Nu";
    return 0;
}
```

Soluție Pascal:

```
program NumerePrietene;
var
    a, b, sa, sb: int64;
    fin: text;

procedure sumap(n_val: int64; var s: int64);
var
    d: int64;
begin
    s := 1;
    d := 2;
    while d * d <= n_val do
    begin
        if n_val mod d = 0 then
        begin
            s := s + d;
            if d * d <> n_val then
                s := s + (n_val div d);
        end;
        d := d + 1;
    end;
end;

begin
```

```

assign(fin, 'DATE.TXT');
reset(fin);
read(fin, a, b);
close(fin);

sumap(a, sa);
sumap(b, sb);
if (sa = b) and (sb = a) then
    writeln(a, ' ', b)
else
    writeln('Nu');
end.

```

SUBIECTUL al II-lea (30 de puncte)

1. Mijloc de învățământ pentru conținutul A: Subprograme recursive

Mijloc ales: mediu de programare cu depanator pas cu pas și vizualizare a stivei apelurilor.

a) Caracteristici și avantaje

- Permite rularea instrucțiunilor pas cu pas, astfel încât elevii pot observa intrarea în subprogram, valorile parametrilor și revenirea din apel.
- Vizualizează stiva apelurilor, ceea ce face concretă ideea de apel recursiv și condiție de oprire.

Avantaje față de o prezentare exclusiv verbală:

- Elevii văd efectiv cum se acumulează apelurile recursive și unde se întorc rezultatele.
- Erorile tipice, precum lipsa condiției de oprire sau modificarea greșită a parametrului, pot fi observate imediat prin rulare.

b) Exemplu de valorificare didactică

- **Activitate de învățare:** calculul recursiv al factorialului.
- **Metodă:** demonstrația combinată cu conversația euristică.
- **Formă de organizare:** frontal pentru prima rulare, apoi individual la calculator.

Scenariu:

- Profesorul pornește de la relația matematică $n! = n * (n - 1)!$ și cere elevilor să identifice valoarea de oprire.
- Profesorul scrie subprogramul `fact(n)` și rulează pas cu pas pentru $n=4$, oprindu-se la fiecare apel pentru a evidenția parametrul curent.
- Elevii notează succesiunea apelurilor `fact(4)`, `fact(3)`, `fact(2)`, `fact(1)` și revenirea valorilor 1, 2, 6, 24.
- Elevii modifică programul pentru suma primelor n numere naturale, aplicând aceeași schemă recursivă.

Exemplu C++:

```

int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}

```

2. Test practic pentru conținutul A: Subprograme recursive

Testul urmărește competențele de înțelegere, urmărire și implementare a subprogramelor recursive.

Item	Enunț	Răspuns așteptat / etape
1	Precizați condiția de oprire pentru un subprogram recursiv care calculează suma cifrelor unui număr natural.	$n = 0$, când nu mai există cifre de prelucrat.
2	Urmăriți apelurile pentru $f(4)$, unde $f(n)=n+f(n-1)$, $f(0)=0$.	$f(4)=4+3+2+1+0=10$.
3	Scriveți un subprogram recursiv care calculează suma primelor n numere naturale.	Antet corect, caz de bază $n=0$, apel $n + \text{suma}(n-1)$.
4	Transformați recursivitatea de la itemul 3 într-o variantă iterativă.	Bucă de la 1 la n , acumulare într-o variabilă s .
5	Explicați de ce un subprogram recursiv fără caz de bază este incorect.	Produce apeluri infinite până la epuizarea stivei; programul nu se termină normal.

SUBIECTUL al III-lea (30 de puncte)

Finalitățile educației

Finalitățile educației reprezintă orientările valorice și proiective ale activității educaționale. Ele indică direcția formării personalității și răspund la întrebarea: ce tip de om și ce tip de competențe urmărește sistemul educațional să formeze?

Clasificare:

- finalități macrostructurale: idealul educațional;
- finalități intermediare: scopurile educației;
- finalități operaționale: obiectivele educaționale formulate pentru lecții sau secvențe concrete.

Idealul educațional exprimă modelul de personalitate dorit de societate: o persoană autonomă, responsabilă, capabilă de integrare socială, profesională și culturală. Are caracter general, orientativ și stabil pe termen lung.

Scopurile educaționale concretizează idealul pe trepte de școlaritate, discipline sau arii curriculare. De exemplu, la informatică, un scop este formarea gândirii algoritmice și a capacității de utilizare responsabilă a tehnologiei.

Obiectivele educaționale sunt enunțuri concrete privind achizițiile elevilor. Ele pot fi cognitive, afective sau psihomotorii. Un obiectiv bine formulat precizează comportamentul observabil, condițiile de realizare și criteriul de performanță.

Proceduri de operaționalizare: Pentru a transforma un scop general într-un obiectiv operațional se folosesc verbe observabile: „definește”, „identifică”, „implementează”, „compară”, „argumentează”. Exemplu: „La finalul lecției, elevul va implementa în C++ un subprogram recursiv pentru calculul factorialului, folosind corect condiția de oprire și apelul recursiv.”

II. Rezolvare Definitivat Informatică de gestiune Varianta 3 (18 iulie 2013)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Formulare în produse pentru gestiunea bazelor de date

Formularul este un obiect al bazei de date folosit pentru introducerea, modificarea și vizualizarea datelor într-o interfață prietenoasă. El nu înlocuiește tabelul, ci oferă o vedere controlată asupra datelor din tabele sau interogări.

Informația dintr-un formular este organizată în antet, zonă de detaliu și subsol. Pot apărea controale precum: casete text, etichete, liste derulante, butoane de comandă, casete de validare și subformulare.

Proiectarea se poate modifica în modul Design/Layout: se schimbă sursa de date, poziția controalelor, etichetele, validările sau evenimentele asociate. Două modalități de creare sunt: folosirea unui asistent, care generează formularul pe baza câmpurilor selectate, și proiectarea manuală, care permite control complet asupra structurii și aspectului.

2. Unitatea centrală de procesare

UCP include unitatea de comandă-control, unitatea aritmetico-logică și registrele interne.

Funcții importante:

- preia și decodifică instrucțiunile din memorie;
- execută operații aritmetice și logice;
- coordonează transferul datelor între memorie, magistrale și dispozitivele sistemului.

3. Subprogramul sumap și verificarea numerelor prietene

Antet C++:

```
void sumap(long long n, long long &s);
```

Program C++:

```
#include <iostream>
#include <fstream>
using namespace std;

void sumap(long long n, long long &s) {
    s = 1;
    for (long long d = 2; d * d <= n; ++d)
        if (n % d == 0) {
            s += d;
            if (d * d != n) s += n / d;
        }
}

int main() {
    ifstream fin("DATE.TXT");
    long long a, b, sa, sb;
    fin >> a >> b;
    fin.close();

    sumap(a, sa);
```

```

sumap(b, sb);
if (sa == b && sb == a)
    cout << a << " " << b;
else
    cout << "Nu";
return 0;
}

```

4. Bază de date pentru companie de automobile

Tabele:

- CLIENT(id_client, nume, prenume, adresa);
- MODEL_AUTO(id_model, denumire_model, pret, putere_motor, tip_combustibil);
- AUTOMOBIL(serie_motor, id_model);
- VANZARE(id_vanzare, id_client, serie_motor, data_vanzare).

Relații:

- MODEL_AUTO 1:M AUTOMOBIL;
- CLIENT 1:M VANZARE;
- AUTOMOBIL 1:1 VANZARE dacă fiecare automobil se vinde o singură dată.

Restricții: serie_motor este unică, prețul și puterea sunt pozitive, id_model și id_client sunt chei străine valide, iar aceeași serie de motor nu poate fi asociată mai multor vânzări active.

Interogări utile:

```

-- clienții care au cumpărat un anumit model
SELECT c.prenume, c.nume, c.adresa, a.serie_motor
FROM CLIENT c
JOIN VANZARE v ON v.id_client = c.id_client
JOIN AUTOMOBIL a ON a.serie_motor = v.serie_motor
WHERE a.id_model = :id_model;

-- modele nevândute
SELECT m.*
FROM MODEL_AUTO m
LEFT JOIN AUTOMOBIL a ON a.id_model = m.id_model
LEFT JOIN VANZARE v ON v.serie_motor = a.serie_motor
WHERE v.id_vanzare IS NULL;

```

SUBIECTUL al II-lea (30 de puncte)

1. Mijloc de învățământ pentru subprograme recursive

Mijloc ales: calculator cu mediu de programare și depanator.

Caracteristici:

- permite rularea pas cu pas a programului;
- afișează valorile variabilelor și succesiunea apelurilor.

Avantaje:

- elevii observă concret stiva apelurilor recursive;
- erorile de oprire sau de transmitere a parametrilor se identifică imediat.

Exemplu de secvență: pentru conținutul **Subprograme recursive**, activitatea de învățare este calculul sumei cifrelor unui număr prin recursivitate. Profesorul scrie funcția, rulează exemplul suma(253), oprește execuția la fiecare apel și cere elevilor să noteze valorile parametrului.

Elevii implementează apoi recursiv calculul produsului cifrelor nenule și compară soluția cu varianta iterativă.

2. Test practic pentru subprograme recursive

Item	Enunț	Răspuns așteptat / etape
1	Identificați cazul de bază pentru calculul recursiv al factorialului.	$n \leq 1$, rezultat 1.
2	Urmăriți apelurile funcției $f(n) = n + f(n-1)$, $f(0) = 0$, pentru $n=3$.	$3+2+1+0=6$.
3	Scrieți o funcție recursivă pentru suma cifrelor unui număr.	Caz de bază $n=0$; apel $n \% 10 + \text{suma}(n/10)$.
4	Modificați funcția pentru a număra cifrele pare.	Test pe ultima cifră și apel pe $n/10$.
5	Explicați de ce lipsește terminarea dacă parametrul nu se apropie de cazul de bază.	Apar apeluri infinite și epuizarea stivei.

SUBIECTUL al III-lea (30 de puncte)

Finalitățile educației sunt orientările care dau sens procesului educativ și stabilesc ce tip de personalitate, competențe și valori urmărește școala să formeze.

Clasificare: ideal educațional, scopuri educaționale și obiective educaționale. Idealul are nivelul cel mai general și exprimă modelul de personalitate dorit social. Scopurile concretizează idealul pentru niveluri, discipline sau cicluri de învățământ. Obiectivele sunt formulări operaționale pentru lecții și activități.

Operaționalizare: un obiectiv corect precizează comportamentul observabil, condițiile de realizare și criteriul de reușită. Exemplu: „La finalul lecției, elevul va scrie corect o funcție recursivă pentru calculul sumei cifrelor unui număr natural, folosind caz de bază și apel recursiv.”